

# The compiler is part of your threat model

Building a confidentiality checker, and being wrong six times

Working notes

All measurements: clang / llc / mlir-opt 17.0.6, reproduced end to end.

# The question

---

Does this compiled artifact reveal a secret to someone not entitled to it?

Some dependence on the secret is *intended* — a service returning nothing derived from its inputs is useless. So the property is release-relative:

$$\forall p, s_0, s_1 : \quad R(p, s_0) = R(p, s_1) \implies \text{Obs}_\Theta(\text{Tr}(P, p, s_0)) = \text{Obs}_\Theta(\text{Tr}(P, p, s_1))$$

The adversary picks the public inputs once and two secrets freely. If they can make *any* observable differ while what they are entitled to stays equal, the program leaks.

**A violation is a pair of executions.** No single run is a counterexample.

# Why a verified compiler does not help

---

CompCert-style correctness says the target computes the same *function*. That is a statement about one execution. Confidentiality is not.

Every one of these preserves the computed function and destroys the property:

- ▶ a conditional move becomes a branch → the observer learns the condition
- ▶ a division is strength-reduced → different operations, different timing
- ▶ a secret-scrubbing store is eliminated → nothing read it
- ▶ an allocation size depends on a secret count

**Refinement preserves 1-safety for free. It does not preserve 2-safety.** Narrowing nondeterminism can turn two indistinguishable runs into two distinguishable ones.

## Four outcomes, and why three would be dishonest

---

| Outcome         | Meaning                               | Obligations                |
|-----------------|---------------------------------------|----------------------------|
| <b>verified</b> | no residual forbidden dependence      | must be none               |
| <b>unsafe</b>   | a leak, with a replayable witness     | must be none               |
| unknown         | required semantics or contract absent | must name what is missing  |
| conditional     | correct if named assumptions hold     | must name every assumption |

The two middle values carry the weight. A two-valued tool must *guess* at a function pointer, a recursive call, an opaque callee, a solver timeout — and a guess toward **verified** is the failure that makes such a tool worthless.

`unknown` is how it fails closed. `conditional` is how it stays honest about hardware facts no IR analysis can reach.

# The finding that reframed everything

---

A program written with an explicit secret branch:

|                                | LLVM IR at -O2      | x86-64 assembly |
|--------------------------------|---------------------|-----------------|
| written with a ternary         | select, no br i1    | je              |
| written with the CT mask idiom | <i>identical IR</i> | je              |
| mask + asm barrier             | no select           | branchless      |

leaky source → clean -O2 IR → leaky binary

Mid-end speculation turns the branch into a select, which reads as a repair.

x86-cmov-converter turns it back — default-on, no profitability check on the memory path.

**aarch64 emits csel and does not leak.**

One artifact. Clean IR. Leaks on one target, safe on another.

# The repair everyone writes does not work

---

## The standard constant-time blend

```
return (v & mask) | (x & ~mask);
```

InstCombine recognises the intent and folds it *back* into a `select` — byte-identical IR to the version written with a ternary. The backend then branches on it.

So the author wrote branchless code, the compiler understood the intent, converted it to the construct being avoided, and a later pass produced the branch being avoided.

**A corpus containing a “fixed” twin is not evidence that the repair works.** Only measuring the emitted code is.

## Every way of carrying policy downward leaks

---

| Carrier                            | Measured failure                                                           |
|------------------------------------|----------------------------------------------------------------------------|
| MLIR <code>sps.*</code> attributes | <code>mlir-translate</code> drops all of them, silently                    |
| LLVM metadata                      | <code>combineMetadata</code> discards unknown kinds on merge               |
| parameter attributes               | dropped by every signature-changing pass                                   |
| opaque marker call                 | no definition $\rightarrow$ no link; definition $\rightarrow$ inlined away |
| debug info                         | explicitly non-semantic; may be dropped                                    |

They are all attempts to make a *policy fact* travel through a pipeline whose passes are obliged to preserve only semantics.

**The pipeline is not misbehaving. It was never told these facts mattered.**

## Marking a declassifier: what survives

---

Split the policy from the implementation. The hash inlines freely; a marker on the *result* carries the release identity.

First attempt — an opaque function — **fails structurally**: a declaration must link, and the moment its definition is co-visible, attribute inference deduces `returned`, the optimizer rewrites uses, and the release edge is severed *while the marker sits there decoratively*.

What works: tied-register inline asm — no symbol at all

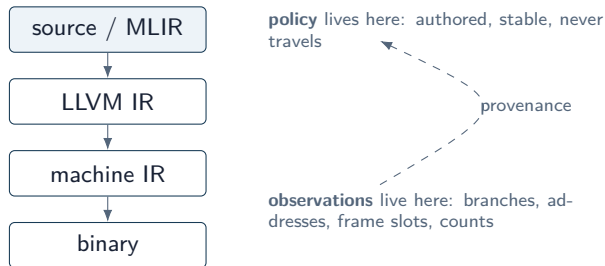
```
%h = call T asm sideeffect "// sps.declassify id=$2",  
    "=r,0,i,~{memory}"(T %raw, i32 <ID>)
```

Nothing to define, link, or import. Survives `01-0z`, `lto`, `thinlto`. **Zero machine cost** (9 instructions vs 9). A non-constant id is a **hard instruction-selection error**.



## If policy cannot go down, send observations up

---



**Level gap closes** — observations gathered where they are real.

**Annotation gap closes** — labels never travel, so nothing can drop them.

**Counting gap closes** — cardinality measured, not asserted.

## ... and that design was wrong too

---

The first version said: *for each policy site, check the predicate over the observations attributed to it.*

That quantifies **existentially over sites**. Soundness needs **universal quantification over observations**.

### Why it matters

A per-site predicate cannot distinguish “no observation here” from “compliant here”. Anything that moves an observation off a named site — outlining, misattribution, a build without `-g` — leaves an empty site that reads as compliance. **It fails open, silently.**

**Universal discharge:** every policy-relevant observation must be discharged by exactly one site. Measured cost: 167 of 168 builds admissible.

*Same mistake one level down* — the first fix rested soundness on the site namespace, which the compiler is equally free to rewrite.

## Then measure it on real code

---

sqlite3.c, 647 functions, -O2 -g:

|                                      |       |
|--------------------------------------|-------|
| Operations lacking a source location | 4.80% |
| Functions containing at least one    | 62.9% |

The fixture corpus said  $\approx 1\%$ . It averages *seven* operations per fixture; real functions have *eighty-nine*. The corpus could never have shown this.

**Refusal granularity is the defect**, not provenance quality: 49% of the refusals are functions missing *one or two* locations out of 89.

*Caveat, stated plainly:* there is no policy here, so every operation was counted as relevant. This is the worst case, not a prediction.

# What actually exists

---

| Artifact                              | Size       | Kind                      |
|---------------------------------------|------------|---------------------------|
| MLIR fixtures, C provenance, examples | ~104       | test data                 |
| check_harness.py                      | 929        | metadata validation       |
| refusal_rate.py                       | 179        | a scanner over IR text    |
| sps-scan.cpp                          | <b>174</b> | <b>an actual analysis</b> |

Of 1,320 lines of code, **174** implement an information-flow analysis. Everything else validates, measures, or documents.

**“The architecture refuses 63% of functions”** is a claim about a system.

**“63% of functions contain an operation with no source location”** is what was measured.

## Running the 174 lines against the corpus

---

| Group                         | Result                                                     |
|-------------------------------|------------------------------------------------------------|
| <b>Caught correctly</b>       | the predecessor-choice countermodel; prefix-causal release |
| <b>Silent correctly</b>       | five fixtures at zero — a blanket-unsafe pass fails here   |
| <b>Imprecise as predicted</b> | the two controls that <i>must</i> lose precision           |
| <b>Missed, known reasons</b>  | audience, release-conformance, helper latency              |

The two firing controls are *not* a defect. Those fixtures exist to document that a forward Low/High lattice must lose precision there, and each declares 11 disposition: relational-required.

**First time a prediction in this work was confirmed by running something** rather than by argument.

## Six claims, six corrections — all by execution

---

| Claim                                                          | Corrected by                                                    |
|----------------------------------------------------------------|-----------------------------------------------------------------|
| whole-run release equality as the product's initial constraint | the spec forbids it; unsound in the <i>permissive</i> direction |
| the marker as an opaque function                               | returned inferred once the definition is co-visible             |
| per-site quantification in the decision                        | a build verified under it, refused under universal discharge    |
| a register-only laundering reduction                           | did not reproduce; needs a memory operand                       |
| an arithmetic-mask repair                                      | folded back to a select, identical to the leaky twin            |
| a refusal surface of $\approx 1\%$                             | 4.8% per observation, 63% per function                          |

None was caught by review, internal consistency, or re-reading the spec. **A design claim that has not been executed is not evidence.**

# The smallest useful next step

---

sps-scan is not referenced anywhere in the harness. Wiring it in turns the corpus from a specification into a test of an implementation.

1. Add it to the diagnostic RUN of the satisfiable gates → real false-negative measurement
2. Add it to the four precision controls → **the first false-positive net the corpus has ever had**
3. Re-derive the refusal number from the analysis, with a real policy

None of this needs the lowering architecture, the .11-versus-MLIR decision, or the thirty unmigrated fixtures. Those gate *later* layers.

# Takeaways

---

- ▶ **The compiler is inside the threat model.** Leaky source, clean IR, leaky binary is reproducible with default flags.
- ▶ **Durable mechanisms ask for nothing.** The inline-asm marker works because it is opaque by construction, not by permission.
- ▶ **Refusal granularity is a soundness *and* usability decision.** Per-function turns 4.8% into 63%.
- ▶ **Corpus size determines what a corpus can tell you.** Seven operations per fixture cannot predict eighty-nine.
- ▶ **The architecture was rewritten twice in a day.** What survived was the reproduced compiler behaviours, the fixtures, and the scripts.

A design that cannot yet be executed should be weighted accordingly — including this one.